

CLARITAS: Explainable AI Misinformation & Trust Detection Engine

Intra IIT Mid Prep Technical Documentation & System Architecture Report

Team Code: 27

September 29, 2026

Abstract

The proliferation of digital misinformation across online news portals and social platforms presents a critical threat to public trust and decision-making. Standard black-box deep learning classifiers often suffer from opaque decision boundaries, poor domain generalization, and vulnerability to subtle linguistic style shifts. In this paper and project submission, we introduce **Claritas**, an end-to-end, privacy-focused Misinformation & Trust Detection Platform. Claritas leverages a **Two-Layer Stacking Ensemble Architecture** combining a fine-tuned DistilRoBERTa transformer (Layer 1) with a calibrated LightGBM meta-classifier (Layer 2) trained on stylistic, sentiment, readability, and source prior feature vectors. Furthermore, Claritas integrates a privacy-preserving **Local Explainable AI (XAI) engine** powered by Ollama/SHAP/LIME, live web content extraction, line-by-line batch pipeline execution, and a persistent Human-In-The-Loop (HITL) audit queue. On the in-domain ISOT test split ($N = 8,980$), Claritas achieves an accuracy of **99.93%**, precision of **100.00%**, recall of **99.87%**, and an AUC-ROC of **0.99999**. We treat this as an in-domain, upper-bound-style result: ISOT is known to contain stylistic shortcuts, and a cross-dataset test (ISOT $\rightarrow$ LIAR, 200 samples) yields only **50.00%** accuracy, which we analyse openly in Section 4.

Contents

1	Introduction & Problem Statement	3
1.1	Problem Statement	3
1.2	Objectives of Claritas	3
2	System Architecture & Pipeline Design	3
2.1	Module 1: Live Web Content Scraper	4
2.2	Module 2: Feature Engineering	4
3	Two-Layer Stacking Ensemble Classification	5
3.1	Layer 1: Deep Fine-Tuned DistilRoBERTa	5
3.2	Layer 2: Calibrated LightGBM Meta-Classifier	5
3.3	Classification Decision Boundaries	5
4	Experimental Results & Model Evaluation	6
4.1	Performance Metrics Table	6
4.2	Confusion Matrix	6
4.3	Cross-Dataset Evaluation (ISOT $\rightarrow$ LIAR)	6
4.4	Failure Case Analysis	7
4.5	Dataset Bias and Validity of the Reported Results	7

5	Explainable AI (XAI) & Privacy-Focused Local LLM Integration	7
5.1	1. SHAP Feature Attribution	7
5.2	2. LIME Token Highlighting	8
5.3	3. Local Ollama LLM Explanation Summaries	8
6	User Interface & Software Architecture	8
7	Installation, Setup & Reproducibility Guide	8
7.1	Directory Structure	8
7.2	Backend Setup Instructions	9
7.3	Frontend Setup Instructions	9
8	Limitations and Responsible Use	9
9	Conclusion & Future Enhancements	10
9.1	Summary of Contributions	10
9.2	Future Work	10

1 Introduction & Problem Statement

1.1 Problem Statement

In modern digital media consumption, false claims, automated propaganda, and sensationalist clickbait spread rapidly before fact-checkers can perform manual verification. Automated misinformation detectors face three major challenges:

1. **The Black-Box Dilemma:** High-performing neural models (e.g., Transformers) provide raw probabilities without explaining *why* a claim was flagged, limiting user trust.
2. **Style vs. Semantic Drift:** Models trained solely on semantics fail when misinformation uses formal news formatting, or when real news contains emotional rhetoric.
3. **Privacy & Operational Overhead:** Offloading explanation generation to cloud LLMs introduces data privacy risks, API cost bottlenecks, and external service dependencies.

1.2 Objectives of Claritas

Claritas is engineered to address these challenges by delivering:

- **Layered Ensemble Classification:** Combining deep stylistic representations with explicit linguistic metrics (polarity, subjectivity, reading ease, casing ratios).
- **Local Explainable AI (XAI):** Delivering SHAP feature attribution bars, inline token highlighting (LIME), and plain-English natural language summaries via locally hosted LLMs without cloud API calls.
- **Live Web Scraping & Batch Processing:** Enabling instant URL content extraction and line-by-line batch claim verification.
- **Human-In-The-Loop (HITL) Audit System:** Providing a persistent audit queue in which reviewer decisions (confirm, dismiss, relabel) are logged as records separate from the original model prediction, so that the original output is preserved for auditability.

2 System Architecture & Pipeline Design

The Claritas pipeline consists of four tightly integrated modules: Data Ingestion & Web Extraction, Feature Engineering, Two-Layer Classification, and Explainable AI / HITL Audit.

High-Level Data Flow Architecture

[User Input: Text / URL / Batch]

↓

[Module 1: Web Scraper & Content Cleaner] (BeautifulSoup4 + Requests)

↓

[Module 2: Feature Extraction Pipeline]

└ *Layer 1*: DistilRoBERTa Semantic & Style Probability (P_{RoBERTa})

└ *Linguistic Metrics*: Polarity, Subjectivity, VADER Compound, Flesch Reading Ease, Caps Ratio

└ *Source Prior*: Bayesian-smoothed source credibility score (S_{domain} ; LIAR speaker-based, global fallback otherwise)

↓

[Module 3: Layer 2 Calibrated LightGBM Meta-Classifier] → Final Verdict & Confidence

↓

[Module 4: Explainability & HITL Audit Engine]

└ SHAP Feature Attribution & LIME Token Highlighting

└ Local Ollama LLM Summary Generation

└ Human Review Queue Persistence

Figure 1: Claritas End-to-End System Pipeline.

2.1 Module 1: Live Web Content Scraper

When a user provides a web URL (e.g., <https://...>), the scraper extracts article content while stripping noise:

1. Fetches page HTML using spoofed browser headers to bypass basic user-agent blocks.
2. Decomposes clutter tags: `<script>`, `<style>`, `<nav>`, `<header>`, `<footer>`, `<aside>`, `<iframe>`.
3. Extracts paragraphs (`<p>`) with character length > 20 . If paragraph length < 150 , falls back to structural body extraction.
4. Triggers a `ScrapingError` if client-side JavaScript rendering blocks plain text extraction.

2.2 Module 2: Feature Engineering

For any cleaned article text T , Claritas computes a 10-dimensional feature vector $\mathbf{x} \in \mathbb{R}^{10}$:

Feature Name	Symbol	Description / Metric Definition
RoBERTa Probability	P_{RoBERTa}	Layer 1 transformer probability score $\in [0, 1]$
Sentiment Polarity	S_{polarity}	TextBlob polarity score $\in [-1.0, 1.0]$
Subjectivity Index	S_{subj}	TextBlob subjectivity score $\in [0.0, 1.0]$
VADER Compound	V_{compound}	NLTK VADER sentiment intensity score $\in [-1.0, 1.0]$
Flesch Reading Ease	F_{flesch}	Readability score (0–100 scale)
Average Word Length	W_{avglen}	Mean character count per word
Capitalization Ratio	R_{caps}	Proportion of uppercase characters in text
Exclamation Ratio	R_{excl}	Frequency of exclamation marks relative to total length
Word Count	N_{words}	Total token count
Source Credibility Score	S_{domain}	Bayesian-smoothed credibility prior $\in [0, 1]$ (see below)

Table 1: 10-Dimensional Feature Vector used by Layer 2 Meta-Classifer.

Source score construction. The source score is computed from the **LIAR training data**, grouped by the **speaker** field, using Bayesian smoothing with a prior strength of 5:

$$\text{source_score}(s) = \frac{\text{mean}(s) \cdot \text{count}(s) + \text{global_rate} \cdot 5}{\text{count}(s) + 5}. \quad (1)$$

Sources that are unseen, or whose records are unavailable, receive the global fallback value. In the ISOT experiments, every article receives this same fallback value (0.438281), so the feature carries no sample-specific information there; it therefore cannot leak labels on ISOT, but it also does not contribute to the ISOT result. A controlled with/without ablation has not yet been run. Consistent with the responsible-use goals of the project, a source score describes a source’s historical pattern and is never treated as proof about an individual article.

3 Two-Layer Stacking Ensemble Classification

3.1 Layer 1: Deep Fine-Tuned DistilRoBERTa

Layer 1 uses a fine-tuned **distilroberta-base** transformer. The input text sequence is tokenized and passed through transformer encoder layers:

$$\mathbf{h}_{\text{CLS}} = \text{DistilRoBERTa}(T) \quad (2)$$

The classification head outputs logits z_0, z_1 , transformed into probability P_{RoBERTa} via Softmax:

$$P_{\text{RoBERTa}} = \frac{\exp(z_1)}{\exp(z_0) + \exp(z_1)} \quad (3)$$

3.2 Layer 2: Calibrated LightGBM Meta-Classifier

The probability P_{RoBERTa} is concatenated with the 9 engineered linguistic and source metrics to construct feature vector $\mathbf{x}$. A LightGBM gradient-boosted decision tree ensemble is wrapped with **isotonic calibration using 5-fold cross-validation** (`CalibratedClassifierCV(method='isotonic', cv=5)`), so that its output approximates a probability:

$$P_{\text{final}} = \mathcal{C}_{\text{isotonic}}(\text{LightGBM}(\mathbf{x})) \quad (4)$$

Sigmoid (Platt) scaling is not used. Calibration quality has not yet been measured with a reliability diagram or Expected Calibration Error (ECE); this is listed as future work.

Training protocol and its limitation. DistilRoBERTa is fine-tuned on the ISOT training split. Its probabilities are then generated for the *entire* ISOT dataset, combined with the engineered features, and the full set is split again into training and test subsets for LightGBM. Consequently, the P_{RoBERTa} values seen by LightGBM are **not out-of-fold** predictions, and many LightGBM training examples were already seen during DistilRoBERTa fine-tuning. The reported stacking metrics may therefore be optimistic (see Section 8).

3.3 Classification Decision Boundaries

The final verdict and confidence score are assigned using the decision thresholds implemented in the `classify()` function and stored in the saved pipeline metadata:

$$\text{Verdict} = \begin{cases} \text{Likely Real,} & \text{if } P_{\text{final}} \leq 0.30 \quad (\text{Confidence} = (1 - P_{\text{final}}) \times 100\%) \\ \text{Likely Misinformation,} & \text{if } P_{\text{final}} \geq 0.70 \quad (\text{Confidence} = P_{\text{final}} \times 100\%) \\ \text{Uncertain,} & \text{if } 0.30 < P_{\text{final}} < 0.70 \quad (\text{Confidence} = (1 - 2|P_{\text{final}} - 0.5|) \times 100\%) \end{cases} \quad (5)$$

These thresholds were chosen to create a deliberately wide “Uncertain” band so that borderline cases are routed to human reviewers. They were not derived from a formal optimisation criterion (e.g., maximising F1 or minimising a cost function).

4 Experimental Results & Model Evaluation

The in-domain evaluation uses a $N = 8,980$ split of the ISOT dataset. Text was cleaned with the notebook’s `deep_clean` function and the `subject` column was removed; no explicit removal of Reuters datelines or other source-specific cues was performed.

4.1 Performance Metrics Table

Evaluation Metric	In-Domain ISOT Test Performance
Accuracy	99.93% (0.99933)
Precision	100.00% (1.00000)
Recall	99.87% (0.99872)
F1-Score	99.94% (0.99936)
AUC-ROC	0.99999 (0.99999881)

Table 2: Quantitative performance on the in-domain ISOT test split ($N = 8,980$).

4.2 Confusion Matrix

		Predicted Class	
		Predicted Real	Predicted Fake
Actual Class	Actual Real	4,284 (True Real)	0 (False Positive)
	Actual Fake	6 (False Negative)	4,690 (True Fake)

Table 3: In-domain confusion matrix on ISOT ($N = 8,980$): 0 false positives and 6 false negatives. Precision and recall here describe ISOT only and should not be read as real-world performance.

4.3 Cross-Dataset Evaluation (ISOT $\rightarrow$ LIAR)

To test generalisation, the fine-tuned DistilRoBERTa model trained on ISOT was evaluated on a random sample of **200 instances** from the LIAR test set. The six LIAR labels were binarised as: `pants-fire`, `false`, `barely-true` $\rightarrow$ Fake (1); `half-true`, `mostly-true`, `true` $\rightarrow$ Real (0).

Metric	Value
Accuracy	0.5000
Precision (macro)	0.5900
Recall (macro)	0.5500
F1 (macro)	0.4500
F1 (weighted)	0.43
AUC-ROC	not computed

Table 4: ISOT $\rightarrow$ LIAR performance (200 LIAR test samples).

Class	Precision	Recall	F1
Real (0)	0.72	0.19	0.30
Fake (1)	0.46	0.91	0.61

Table 5: Class-wise ISOT $\rightarrow$ LIAR results.

Accuracy falls from 99.93% (in-domain) to 50.00%, which is no better than chance on this sample. The class-wise results show that the model labels most LIAR statements as fake (fake recall 0.91) and misses most real ones (real recall 0.19). Likely causes are (i) a change in text type, from long articles to short one-sentence political statements, which distorts length-, readability- and casing-based features; (ii) a change in label semantics, from source reliability in ISOT to claim-level veracity in LIAR; and (iii) style shortcuts learned from ISOT that do not transfer. With only 200 samples the 95% confidence interval on accuracy is roughly ± 7 percentage points, so the result should be re-run on the full LIAR test set.

4.4 Failure Case Analysis

On the ISOT test split the model produced no false positives and six false negatives (fake articles predicted real). The following are three of the false negatives reported by the notebook (texts truncated; p is the predicted fake probability):

Text excerpt	p (fake)
“Because every American Ivy League school needs ...”	0.2251
“Sen. John McCain is weighing in on Russia s me ...”	0.4555
“The fact of the matter is Kenya and the U.S. s ...”	0.4995

Table 6: ISOT false-negative examples.

A qualitative error analysis of these cases has not yet been carried out, and individual LIAR false positives and false negatives were not extracted in the current notebook. Both are planned as part of re-running the LIAR evaluation.

4.5 Dataset Bias and Validity of the Reported Results

ISOT combines real articles from largely one wire service with fake articles from unreliable sites, so a classifier can reach very high accuracy by learning source-specific cues (datelines, formatting, punctuation, topic mix). Our ISOT→LIAR result is direct evidence that part of the in-domain score reflects such cues. For this reason the 99.93% figure is reported as an in-domain, upper-bound-style result and not as evidence that the model generalises to independently sourced news.

5 Explainable AI (XAI) & Privacy-Focused Local LLM Integration

Claritas provides three levels of explainability:

5.1 1. SHAP Feature Attribution

Tree SHAP computes exact Shapley values ϕ_i for each metric x_i in Layer 2:

$$\phi_i(\mathbf{x}) = \sum_{S \subseteq F \setminus \{i\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f(S \cup \{i\}) - f(S)] \quad (6)$$

The relative weight percentage of each metric is rendered in the UI as signal strength indicators.

5.2 2. LIME Token Highlighting

Local Interpretable Model-agnostic Explanations (LIME) perturbs token sequences to compute individual word weights:

$$w_t = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (7)$$

Tokens pushing toward misinformation are highlighted in red, while tokens supporting factual credibility are highlighted in green.

5.3 3. Local Ollama LLM Explanation Summaries

To preserve user privacy and avoid cloud API costs, Claritas connects locally to an **Ollama** instance running Llama 3. It formats metric contributions and LIME weights into a structured prompt, returning plain-English bullet points explaining the decision.

6 User Interface & Software Architecture

The frontend is built with React.js (Vite) and Tailwind CSS, following a component-driven architecture:

Frontend Component Hierarchy

```
App.jsx (React Router)
├─ Navbar.jsx (Sticky navigation header)
├─ AnalysisContext.jsx (Global Context provider preserving state across tab switches)
├─ AnalyzerPage.jsx (Main single/batch input, extracted URL text collapse preview, SHAP/LIME charts)
├─ VerdictCard.jsx (Verdict badge, confidence percentage, LIME token display)
├─ EvidencePanel.jsx (SHAP breakdown & interactive LIME trigger)
├─ HumanReviewModal.jsx (HITL audit override modal)
├─ ReviewQueuePage.jsx (Filterable/sortable audit history table)
├─ HowItWorksPage.jsx (System metrics, confusion matrix, interactive architecture diagram)
```

Figure 2: React Component Architecture.

7 Installation, Setup & Reproducibility Guide

7.1 Directory Structure

```
claritas/
|-- data/
|   '-- domain_history.json
|-- src/
|   |-- backend/
|   |   |-- models/ (DistilRoBERTa & LightGBM weights)
|   |   |-- classifier.py
|   |   |-- main.py
|   |   |-- metrics.py
|   |   '-- scraper.py
|   |-- components/
|   |   |-- AnalyzerPage.jsx
|   |   |-- HowItWorksPage.jsx
|   |   |-- HumanReviewModal.jsx
```

```
| | |-- ReviewQueuePage.jsx  
| | '-- VerdictCard.jsx  
| |-- App.jsx  
| '-- main.jsx  
|-- package.json  
'-- vite.config.js
```

7.2 Backend Setup Instructions

```
# 1. Navigate to backend directory  
cd claritas/src/backend  
  
# 2. Create and activate Python virtual environment  
python3 -m venv venv  
source venv/bin/activate  
  
# 3. Install required Python packages  
pip install fastapi uvicorn torch transformers lightgbm pandas \\\  
          textblob textstat nltk beautifulsoup4 requests joblib shap lime  
  
# 4. Start FastAPI backend server  
python main.py
```

The FastAPI service starts at <http://localhost:8000>.

7.3 Frontend Setup Instructions

```
# 1. Navigate to claritas project root  
cd claritas  
  
# 2. Install NPM dependencies  
npm install  
  
# 3. Launch Vite development server  
npm run dev
```

The React application starts at <http://localhost:5173>.

8 Limitations and Responsible Use

- **Non-out-of-fold stacking.** LightGBM is trained on DistilRoBERTa probabilities for data that DistilRoBERTa had already seen during fine-tuning. Stacking metrics may be optimistic; a leakage-resistant k -fold out-of-fold procedure is required for an unbiased estimate.
- **Calibration unverified.** Isotonic calibration is applied, but no reliability diagram or ECE has been computed. Calibration is expected to degrade under distribution shift such as ISOT→LIAR.
- **Scope.** English text only; static HTML scraping only (JavaScript-rendered pages raise a `ScrapingError`).

9 Conclusion & Future Enhancements

9.1 Summary of Contributions

In this project, we engineered **Claritas**, an end-to-end Explainable AI Misinformation Detection Engine. Key achievements include:

- Achieving **99.93% accuracy** and **100.00% precision** on the in-domain ISOT test split using a two-layer stacking ensemble, while explicitly documenting that cross-dataset accuracy on LIAR is substantially lower (50.00% on 200 samples).
- Addressing the black-box dilemma via local SHAP attribution, LIME token highlights, and local Ollama LLM summaries.
- Eliminating state resets during live web scraping by isolating persistence files outside the Vite HMR directory.
- Providing full batch processing and persistent Human-In-The-Loop audit capabilities.

9.2 Future Work

- **Calibration and evaluation:** Reliability diagrams and ECE, threshold optimisation, full LIAR test-set evaluation with AUC, and multi-dataset training (ISOT, LIAR, Fake-NewsNet).
- **Multimodal Verification:** Extending Layer 1 to accept synthetic image detection features alongside text.
- **Active Learning Auto-Tuning:** Automatically re-triggering LightGBM calibration steps as human audit feedback accumulates in the review queue.

— End of Documentation —